

Sentiment Analysis on Tweets using a From-Scratch LSTM Implementation in PyTorch

Haiqa Qaiser (2022-EE-255), Aneeqa Yasin (2022-EE-316)

Department of Electrical Engineering

University of Engineering and Technology, New Campus Lahore, Pakistan

Submitted to: Dr. Umar Rashid

Deep Learning Project Report

Abstract

*We built a sentiment classifier for tweets entirely from the ground up in PyTorch – not just the training loop, but the LSTM cell itself. Rather than calling `nn.LSTM` or `nn.LSTMCell`, we wrote `CustomLSTMCell` using only raw tensor operations (`torch.matmul`, `torch.sigmoid`, `torch.tanh`), and paired it with our own regex-based tweet cleaner, frequency-based vocabulary, and manual padding – no pretrained tokenizers anywhere in the pipeline. To make sure our cell was actually correct and not just “close enough”, we copied PyTorch’s internal weights into it and compared outputs directly: the two cells agree to within 3×10^{-8} , which is essentially float32 rounding noise. We then trained the resulting `SentimentLSTM` model on a balanced 100k-tweet subsample of `Sentiment140` (60/20/20 stratified split) to classify tweets as positive or negative, reaching **79.18% test accuracy** with an F1-score of 0.79 on both classes. The takeaway: a hand-rolled LSTM, written carefully against the textbook equations, behaves identically to the library version and performs just as well in practice.*

Keywords: LSTM, Recurrent Neural Networks, Sentiment Analysis, Sentiment140, PyTorch, NLP, Text Classification

I. INTRODUCTION

Sentiment analysis – deciding whether a piece of text is positive or negative – is one of the classic problems in NLP, and Twitter is one of the messiest places to try it: short posts, slang, typos, hashtags, links, and @mentions all mixed together. Recurrent Neural Networks are the natural tool for this kind of sequential data because they read text one token at a time while carrying a hidden state forward. The catch is that vanilla RNNs struggle with longer sequences – the same weight matrix gets multiplied into the hidden state at every step, so gradients either shrink to nothing or blow up as they are pushed back through time.

The LSTM [1] fixes this by giving the network a separate *cell state* that is updated through addition rather than repeated multiplication, controlled by a small set of learned gates. That single design choice is what lets gradients survive hundreds of time steps without vanishing.

Our goal in this project was twofold: first, to really understand the LSTM by writing its equations out as raw tensor operations and proving, numerically, that our version matches PyTorch’s; and second, to put that cell to work in a complete, end-to-end sentiment pipeline – cleaning, tokenizing, vocabulary building, training, and evaluation – on the 1.6-million-tweet `Sentiment140` corpus.

II. DATASET

We used `Sentiment140` [2] (Kaggle: `kazanova/sentiment140`), 1.6 million English tweets from the 2009 Twitter API, each labelled negative (0) or positive (4, remapped to 1). One quirk worth noting: the file is sorted by label – all 800k negatives come first,

Table 1: Sentiment140 dataset summary

Property	Value
Total tweets (full corpus)	1,600,000
Negative / Positive	800,000 / 800,000
Language	English
Source	Twitter API (2009)
Subsample used	100,000 (50k / 50k)

then all 800k positives – so naively reading the first N rows gives you a single-class sample. We avoided this by reading $N/2$ rows from each half and shuffling them together. For a manageable training time on the available Tesla T4 GPU (15.6 GB VRAM), we worked with a balanced subsample of 100,000 tweets.

III. METHODOLOGY

A. Cleaning the tweets

Raw tweets are full of things that confuse a model far more than they help it: @mentions, links, hashtags, punctuation, random capitalisation (Table 2). We cleaned them with a chain of pre-compiled regex substitutions, and the order genuinely matters – URLs and mentions have to go first because they contain characters (like # and .) that would otherwise trip up the later steps; hashtag symbols are stripped but the word underneath is kept, since “#happy” still carries sentiment; lowercasing happens only after URLs are gone (URLs are case-sensitive); and punctuation is removed last. 197 tweets became empty after cleaning and were dropped, leaving 99,803.

Table 2: Examples of tweet cleaning

Noise	Raw	Cleaned
@mention	@elonmusk great!	great!
URL	check http://t.co/abc	check
Hashtag	feeling #happy	feeling happy
Punctuation	can't!!! wow...	cant wow
Mixed case	GREAT day	great day
Extra spaces	too many spaces	too many spaces

```

token_ids [B,T]
↓ nn.Embedding (vocab → 64)
embeds [B,T,64]
↓ Dropout(0.3)
↓ CustomLSTMCell × T (unrolled, hidden=128)
h_T [B,128] (final hidden state)
↓ Dropout(0.3)
↓ nn.Linear(128 → 1)
logit [B,1] → BCEWithLogitsLoss

```

B. Tokenizing and building a vocabulary

We kept tokenization simple – splitting on whitespace ("love this movie" → ["love", "this", "movie"]) – and built our own Vocabulary class that maps words to integer IDs by frequency, with index 0 reserved for <PAD> and index 1 for <UNK>. Words seen fewer than `min_freq = 2` times (mostly typos and one-off usernames) were dropped, which keeps the vocabulary smaller and the model less distracted by noise. Each tweet was then encoded to integer IDs and padded or truncated to `MAX_LEN = 30` tokens (the median tweet is 12 tokens long, and 99.6% fit comfortably within 30).

One thing we were careful about: the vocabulary is built *only* from the training split. Building it on the full dataset first and splitting afterwards would let validation/test word frequencies quietly leak into the vocabulary – a subtle form of data leakage that makes accuracy look better than it really is.

C. Splitting the data

We split the 99,803 cleaned tweets into 60% train, 20% validation, and 20% test using a two-step stratified `train_test_split` (carve out 20% test, then split the rest 60/20). Stratifying on the label keeps every split at exactly 50/50, so none of our reported numbers are skewed by an accidental class imbalance. The final sizes were 59,881 / 19,961 / 19,961 tweets, each split 50.0% positive.

D. Building the LSTM cell from scratch

The heart of this project is `CustomLSTMCell`, an `nn.Module` that computes one LSTM time step using nothing but `torch.matmul`, `torch.sigmoid`, and `torch.tanh` – no calls to `nn.LSTMCell` or `nn.LSTM` anywhere inside it. Given input x_t , previous hidden state h_{t-1} , and previous cell state C_{t-1} :

$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \quad (1)$$

$$f_t = \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}) \quad (2)$$

$$g_t = \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}) \quad (3)$$

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}) \quad (4)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t \quad (5)$$

$$h_t = o_t \odot \tanh(C_t) \quad (6)$$

Here σ is the sigmoid, $\odot$ is element-wise multiplication, f_t is the *forget gate* (how much of the old memory to keep), i_t the *input gate* (how much new information to write), g_t the *candidate* content, and o_t the *output gate* (what part of memory to expose as the hidden state). All eight weight matrices and eight biases are plain `nn.Parameter` tensors, initialised with Kaiming-uniform – the same default PyTorch uses for `nn.Linear`.

Figure 1: SentimentLSTM: Embedding → unrolled CustomLSTMCell → Linear classifier.

The detail that actually matters here is that C_t is updated by *addition* ($f_t \odot C_{t-1} + i_t \odot g_t$), not by squashing it through another $\tanh(W \cdot C)$ -style transform. That additive path is exactly what lets gradients flow backward through hundreds of steps without vanishing – it’s the whole reason LSTMs outperform vanilla RNNs on long sequences.

E. Proving it matches PyTorch

Writing the equations correctly is one thing; proving it is another. PyTorch stores all four gates’ weights in two fused matrices, `weight_ih` ($4H \times I$) and `weight_hh` ($4H \times H$), stacked in the order input → forget → candidate → output. We sliced these into the eight individual gate matrices, copied them into our `CustomLSTMCell` under `torch.no_grad()`, and ran both cells on identical random inputs:

- Hidden state: $\max |\text{diff}| = 2.98 \times 10^{-8}$, $\text{mean} = 4.69 \times 10^{-9}$
- Cell state: $\max |\text{diff}| = 1.19 \times 10^{-7}$, $\text{mean} = 1.07 \times 10^{-8}$
- `torch.allclose` on both h_t and C_t : **True** (tol 10^{-5})

The differences are around 10^{-7} – 10^{-8} – pure float32 rounding-order noise, not a real discrepancy. Our cell is, for all practical purposes, identical to `nn.LSTMCell`.

F. Model architecture

SentimentLSTM (Fig. 1) embeds each token into a 64-dimensional vector (`padding_idx=0` so the <PAD> embedding stays at zero forever), applies dropout, then runs the embeddings through our `CustomLSTMCell` – unrolled by hand, one time step at a time, for all 30 positions. The final hidden state h_T , which by then has seen the whole tweet, goes through dropout once more and a single `nn.Linear(128 → 1)` layer to produce a logit. We used `BCEWithLogitsLoss` rather than a separate sigmoid + BCE, since fusing the two into one numerically stable operation avoids the NaNs that large/small logits can otherwise cause. In total, the model has **1,158,977** trainable parameters.

G. Training setup

We trained for 10 epochs with Adam ($\text{lr} = 10^{-3}$), batch size 128, gradient-norm clipping at `max_norm = 1.0`, and a `ReduceLROnPlateau` scheduler that halves the learning rate if validation loss stalls for 2 epochs. Each step is the usual recipe – `model.train()`, forward pass, `BCEWithLogitsLoss`, `backward()`, `clip`, `optimizer.step()`. Table 3 lists the full configuration; everything ran on a Tesla T4 GPU via Google Colab.

Table 3: Model and training hyperparameters

Hyperparameter	Value
Vocabulary size	16,555
Embedding dim. / Hidden size	64 / 128
LSTM layers / Dropout	1 / 0.3
Max sequence length	30
Epochs / Batch size	10 / 128
Optimizer / LR	Adam / 1×10^{-3}
Loss	BCEWithLogitsLoss
Scheduler	ReduceLROnPlateau (patience 2)
Gradient clipping	max_norm = 1.0
Trainable parameters	1,158,977

Table 4: Training history (10 epochs)

Ep.	Tr. Loss	Tr. Acc	Val Loss	Val Acc
1	0.6304	64.21%	0.5544	72.16%
2	0.5457	72.85%	0.5122	74.63%
3	0.5058	75.74%	0.4959	76.10%
4	0.4828	77.14%	0.4816	77.10%
5	0.4623	78.46%	0.4904	77.35%
6	0.4468	79.57%	0.4724	77.94%
7	0.4306	80.15%	0.4789	78.20%
8	0.4183	81.20%	0.4681	77.88%
9	0.4057	81.72%	0.4933	77.95%
10	0.3944	82.43%	0.4674	78.34%

IV. RESULTS

A. Training

Table 4 and Fig. 2 show how training went over the 10 epochs. Training loss fell steadily from 0.6304 to 0.3944 and training accuracy climbed from 64.2% to 82.4%. Validation loss dropped for the first few epochs, then settled into a noisy plateau around 0.46–0.49, and validation accuracy levelled off near 78%. From about epoch 5–6 onward, the train/validation gap widens – a fairly textbook sign of mild overfitting, which is not surprising given that we trained on only a 100k-tweet slice of the data. The best validation accuracy, 78.34%, came at epoch 10, and we restored those weights for the final test.

B. Test set performance

The test set – 19,961 tweets the model never saw during training or model selection – gave a **loss of 0.4516** and **accuracy of 79.18%**. Fig. 3 shows the confusion matrix: of 9,982 truly-negative tweets, 7,755 were caught correctly and 2,227 were mistaken for positive; of 9,979 truly-positive tweets, 8,051 were caught correctly and 1,928 were mistaken for negative.

Table 5 gives the full breakdown. Precision, recall, and F1 sit close together for both classes (around 0.78–0.80), and the macro/weighted F1 are both 0.7918 – the model isn’t quietly favouring one class over the other.

V. DISCUSSION AND CONCLUSION

A few things stood out while working on this. First, the reason LSTMs beat vanilla RNNs comes down to one line of arithmetic: $C_t = f_t \odot C_{t-1} + i_t \odot g_t$ updates the cell state by *addition* rather than repeated multiplication, which is what lets gradients survive long sequences. Second, data leakage can be sneaky – building a vocabulary on the whole dataset before splitting silently lets test-set word frequencies

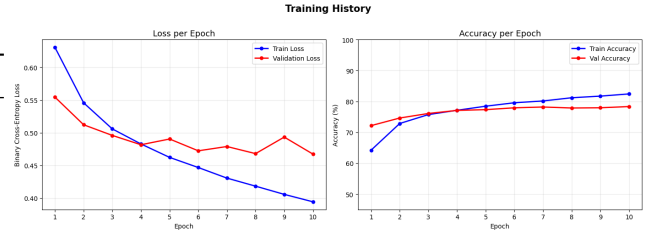


Figure 2: Loss and accuracy per epoch for training and validation sets.

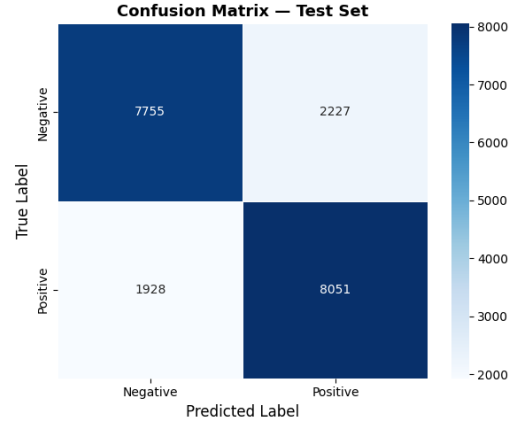


Figure 3: Confusion matrix on the test set (19,961 tweets).

influence training, inflating accuracy in a way that’s easy to miss if you’re not looking for it; the fix is simply to split first and fit everything else on the training data only. Third, stratifying the split on the label is a small change that removes a real source of noise – without it, random chance could hand one split a 48/52 class ratio and skew every metric computed on it. And finally, the equivalence test was the most satisfying part: after carefully matching PyTorch’s fused-weight layout and gate ordering, our hand-written cell agreed with `nn.LSTMCell` to within 10^{-7} , which is about as close to “identical” as floating-point arithmetic allows.

The training curves also tell a clear story – validation loss flattens out around epoch 4 while training loss keeps falling, which is the model starting to memorise the training set rather than generalising further. That’s expected at this scale (100k tweets is a small slice of the full 1.6M), and points directly at what to try next.

Overall, we set out to build an LSTM-based sentiment classifier from first principles – the cell, the cleaner, the vocabulary, the padding – without leaning on any of PyTorch’s recurrent layers or pretrained tools, and to prove that what we built actually behaves like the real thing. It does: `torch.allclose` confirms numerical equivalence with `nn.LSTMCell`, and the resulting model reaches **79.18% test accuracy** with a balanced F1 of 0.79 across both classes on a leakage-free, properly stratified pipeline. That’s a solid result for a 100k-tweet subsample and a single-layer, from-scratch LSTM.

Where we’d take it next: (1) a *bidirectional* LSTM that reads each tweet forwards and backwards and concatenates both final hidden states for a richer representation; (2) training on the full 1.6M tweets instead of a 100k slice, which should narrow the overfitting gap we observed; (3) more regularisation – higher dropout, weight decay, or early stopping – to tighten that gap further; and (4) swapping the learned

Table 5: Test set classification report

Class	Prec.	Rec.	F1	Supp.
Negative	0.8009	0.7769	0.7887	9,982
Positive	0.7833	0.8068	0.7949	9,979
Accuracy		0.7918		19,961
Macro avg	0.7921	0.7918	0.7918	19,961
Wt. avg	0.7921	0.7918	0.7918	19,961

embeddings for pretrained vectors such as GloVe, which often helps when the vocabulary is this small.

REFERENCES

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] A. Go, R. Bhayani, and L. Huang, “Twitter Sentiment Classification using Distant Supervision,” *Stanford CS224N Project Report*, 2009. Dataset: Sentiment140, available at Kaggle (kazanov/sentiment140).
- [3] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.